MLOps Assignment: Debugging Machine Learning Pipelines with Python Debugger (PDB)

Assignment Overview

In this assignment, you'll use the Python Debugger (**PDB**) to identify and fix issues in a machine learning pipeline. The focus will be on troubleshooting typical errors in data processing, model training, and evaluation by using PDB to pause execution, inspect variables, and understand the code's flow.

- You can use Google colab for this assignment.

Assignment Details

Tasks:

1. **Fix Errors in Data Preprocessing**: Identify and correct data issues in a preprocessing function.
2. **Debug the Model Training Process**: Use PDB to inspect model parameters, data shapes, and loss values during training.
3. **Analyze Evaluation Issues**: Diagnose and resolve issues in the model evaluation phase, using PDB to inspect predictions and metric calculations.

Data Preprocessing Debugging:

The following code preprocesses data by scaling and normalizing it for model training. However, it **contains some bugs**.

```python
import pdb
import numpy as np
from sklearn.preprocessing import StandardScaler

def preprocess_data(data):
    scaler = StandardScaler()
    pdb.set_trace()  # Start debugger here
    scaled_data = scaler.fit_transform(data)
    normalized_data = scaled_data / np.linalg.norm(scaled_data)
    return normalized_data

# Example Data
data = np.array([[1, 2, 3], [4, 5, np.nan], [7, 8, 9]])

processed_data = preprocess_data(data)
print("Processed Data:\n", processed_data)
```

1. **Task 1.1**: Use PDB to inspect the `data` variable and identify why this preprocessing step fails. Implement a fix that handles missing values by replacing them with the mean of the column. Set a breakpoint with `pdb.set_trace()` at the beginning of the function, then use `p` and `n` commands **explained in the tutorial sheet** to debug. **[1.5]**
   Task 1.2: Write down the corrected version of the `preprocess_data()` function after fixing the bug. **[1.5]**

Model Training Debugging:
The following code trains a simple linear regression model using scikit-learn. However, there are issues with **mismatched shapes** and **unexpected training outcomes**. Use PDB to identify and fix the issues.

```python
from sklearn.linear_model import LinearRegression
import numpy as np

def train_model(X, y):
    model = LinearRegression()
    pdb.set_trace()  # Debugger for model training
    model.fit(X, y)
    return model

# Example Data
X = np.array([[1], [2], [3], [4]])
y = np.array([1, 4, 9, 16, 25])  # Incorrect shape on purpose

trained_model = train_model(X, y)
print("Trained Model Coefficients:", trained_model.coef_)
```

2. **Task 2.1**: Start the debugger and use `p X.shape` and `p y.shape` to check the shapes of `X` and `y`. Identify the error causing the shape mismatch.
   Task 2.2: Implement a fix by adjusting the shape of `y` and update the `train_model()` function accordingly. Once the updates are complete, rerun the cell to start the debugging process again from the beginning. **[2]**
3. **Task 2.3**: After fixing, use `n` to move through the code and inspect the values of `model.coef_` to ensure they are correct. **[2]**
4. Then use `c` to continue the execution of the function and get the final output.

Model Evaluation Debugging:

In this part, you will debug an evaluation function that calculates the Mean Squared Error (MSE). However, it produces an **unexpectedly high error**.

```python
from sklearn.metrics import mean_squared_error
import numpy as np

def evaluate_model(model, X_test, y_test):
    predictions = model.predict(X_test)
    pdb.set_trace()  # Debugger for evaluation
    mse = mean_squared_error(y_test, predictions)
    return mse

X_test = np.array([[5], [6], [7], [8]])
y_test = np.array([25, 36, 49, 64])

# Using trained_model from previous task
mse_score = evaluate_model(trained_model, X_test, y_test)
print("Mean Squared Error:", mse_score)
```

5. **Task 3.1**: Use PDB to inspect the `predictions` and `y_test` arrays and identify why the error might be higher than expected. **[1.5]**
   Task 3.2: Adjust the training data or model parameters in the training function if necessary. Re-run the code to ensure the MSE calculation is accurate. **[1.5]**

Submission

- Submit a Python script (colab) with the final, debugged versions of each function.
- Include comments or a brief document describing how you used PDB commands in each section and the issues you identified in the colab file itself.